Attacking and Defending Image Immunisation: Adversarial Gameplay against PhotoGuard and ESD-x

Wanting Zhao (35507071)
FIT5230 Malicious AI, Milestone 4 · Team Dark.Tyro (Theme 2, Text-to-Image, Dark side) · Monash University

PhotoGuard [1] protects a photograph from diffusion-based editing with an imperceptible adversarial perturbation. IMPRESS [2] removes it through an optimisation loop that restores consistency under Stable Diffusion's autoencoder. I worked on the attacking side: IMPRESS was our baseline attack against PhotoGuard, and my individual strategy tested another team's ESD-x concept-erasure defence [3]. This report logs my strategy for Weeks 2–12, recaps the system developed in Milestones 1–3, and presents two new demonstrations. The first replaces IMPRESS's 100-step optimisation with five-step masked diffusion purification (version D). Compared with IMPRESS, D caused 62% less photo damage and ran about 93× faster per face. It missed the criterion fixed before the run because it caused 19% more damage than our masked IMPRESS variant. The second demonstration defends PhotoGuard by detecting purified photographs. The pre-specified detector flagged none. Post hoc analysis found that IMPRESS-purified images had about nine times the VAE reconstruction error of clean images. This separated all ten purified images from all fifty clean images. Masked purification produced no such signal. The effect's direction was identified after data inspection, so it remains an observation, not a confirmed result.

***Index Terms*—Adversarial purification, diffusion models, image immunisation, PhotoGuard, IMPRESS, concept erasure.**

I. Introduction

DIFFUSION
iffusion-based editors such as Stable Diffusion inpainting [6] can turn one real photograph into a convincing fake within seconds. PhotoGuard [1] counters this misuse with a small perturbation that misleads the editor's image encoder and distorts later edits. IMPRESS [2] attacks PhotoGuard. It finds a nearby image that Stable Diffusion's variational autoencoder (VAE) reconstructs consistently, removing the protection. Here, *protection* means PhotoGuard's perturbation; *purification* means any process that removes it. IMPRESS was our baseline paper, and PhotoGuard was our target defence.

Our team joined the Dark (attack) side of Theme 2. From Milestone 1 (M1) to M3, our pipeline used four stages—*protect, purify, edit,* and *score*—with ten fixed faces from the Helen dataset [12], one editor, one prompt, and one inpainting mask. We used two metrics:

- **Photo damage**: LPIPS [7] between the purified photo and the clean photo (lower is better). On identical inputs it reproduces to three decimals.
- **Recovery**: $R_{\text{pipe}} = (\text{SSIM}_{\text{pur}} - \text{SSIM}_{\text{prot}})/(1 - \text{SSIM}_{\text{prot}})$, where each SSIM [8] compares an edit with the edit of the clean photo (higher is better). Its measured run-to-run noise is about ±3 percentage points (pp).

For my individual M3 strategy I targeted Light team #14 (Cyber Ninjas), whose project erases Van Gogh's painting style from Stable Diffusion using ESD-x [3].

Contributions. This report provides (1) a weekly strategy log for Weeks 2–12 (Section III); (2) an unconstrained system enhancement supported by a proof of concept that replaces IMPRESS's optimisation loop with masked diffusion purification (Section IV-A); (3) a role-reversal strategy for PhotoGuard, supported by a detector of purified photos (Section IV-B); and (4) a reflection on how my background shaped these decisions (Section IV-C). I fixed each pass/fail criterion before the run. Both demonstrations failed. Their failures and causes form part of the result.

II. System Recap (M1–M3)

A. Baseline and Provenance

IMPRESS's purification algorithm (`impress.py`, about 25 lines) belongs to its authors. We kept it unchanged as a faithful reference. Our work added (i) eleven compatibility repairs for the 2026 software stack, covering a removed model repository, a missing dependency, a hard-coded four-GPU configuration, and an unsupported GPU architecture; (ii) an evaluation harness that scores each pipeline stage on fixed faces, seeds, and settings; and (iii) the purification change described next.

B. Masked Purification (M2)

An inpainting editor *repaints* the region inside its mask and *keeps* the region outside. Between a clean photo and its edit, pixels in the kept region changed by only 4–7 grey levels; those in the repainted region changed by 86–96. The editor also blanks the repainted region before encoding, discarding any purification there. I restricted purification to the kept region. Let x_p be the protected image, $P(\cdot)$ the IMPRESS purifier, and $m \in \{0,1\}^{H\times W}$ the editor's inpainting mask (1 = repainted). The masked variant is

$$x_C = (1 - m) \odot P(x_p) + m \odot x_p, \tag{1}$$

where $\odot$ denotes element-wise multiplication (the code feathers the mask edge with a 3-pixel Gaussian blur). The purifier changes only the region where the edit depends on the original image. Protected pixels remain unchanged where the editor will overwrite them.

TABLE I
THE CANONICAL M3 RESULT ($n = 10$ FACES; PROTECTION AT L2, EPS 16, STEP 1, 40×2 ITERATIONS; ONE SHARED PROTECTION STAGE)

Version	Purif. iters	Damage (LPIPS) ↓	Recovery (pp) ↑
N · no purification	0	0.034	0.0
A · IMPRESS	100	0.185	+0.2
B · IMPRESS	1000	0.143	+4.9*
C · A + mask (1)	100	**0.060**	+0.9

*B's gain exceeds the ±3 pp noise band, but two of the ten faces carry it (+29 and +25 pp; median +3.0 pp). The evidence is suggestive, not established.



Fig. 1. M3 edits for four of the ten faces (prompt "a person in an airplane"). Columns: clean photo, edit of the clean photo (target), edit of the protected photo (N), and edits after purification by A and C, labelled with photo damage and recovery.

C. Canonical M3 Result

Version C caused **68% less photo damage than A at the same compute** and 58% less than B at one-tenth of B's compute. Its recovery was within A's noise band and 3.9 pp below B's. The result shows a trade-off: **B achieved the highest recovery; C preserved the photo best.** Protection was weak in all versions. The editor's output changed noticeably on 0 of 10 faces because 40×2 iterations used one-twenty-fifth of PhotoGuard's reference effort (200×10).

D. Individual Strategy: Attacking ESD-x (M3)

The defence. Erased Stable Diffusion (ESD) [3] removes a concept from a text-to-image model by fine-tuning it against itself. When the prompt contains the target concept, the fine-tuned model is trained to predict the noise that the original model would predict *with the concept pushed away*, a form of negative classifier-free guidance. The ESD-x variant fine-tunes only the cross-attention layers, where the text prompt enters the image model. This makes the erasure tied to the words of the prompt and limits collateral damage to unrelated concepts. Cyber Ninjas applied ESD-x to erase Van Gogh's style from Stable Diffusion v1-4. Their own analysis acknowledged the open question for any word-level erasure: an attacker may reach the same style without using the erased name, and their stated next step was to train on several descriptors of the style.

The attack. The erasure used only the English name during training. I tested uncovered routes: the name in five other scripts (Chinese, Japanese, Korean, Russian, and Arabic), Van Gogh painting titles, and a style description without his name. I generated each route on the erased model and unmodified SD v1-4 over six seeds. CLIP ViT-L/14 [9] scored the outputs against one fixed reference sentence ("a painting by Vincent van Gogh"). A route counted as a bypass only if the unmodified model reliably drew the style and the erased model retained that signal.

The result. No route restored Van Gogh's style. The non-Latin names gave no signal, because the unmodified model never drew Van Gogh from them, so there was nothing to bypass. The painting titles gave weak and unreliable signal on the unmodified model as well. The description and title-plus-description routes were erased (−4% and 27% of the original signal retained). A Gauguin control retained 105%, showing that the erasure is specific to Van Gogh. Two limitations remain. First, CLIP scores the painting as a whole: by eye, on one seed, the erased model still painted a swirling starry sky when the prompt described one. Second, the authors' checkpoint differs from our copy of SD v1-4 outside the cross-attention layers almost as much as inside them, so the erasure cannot be fully separated from differences in the starting weights.

III. WEEKLY STRATEGY LOG (WEEKS 2–12)

Each entry records the strategy I chose, the advantage I expected from it, and how the outcome shaped my next step. The full diary is kept separately.

Week 2 (3–9 Aug) · Taking ownership of the plan. I registered our team on the Dark side early and set up a shared Drive with introductory material. This gave my teammate and me the same foundation. I read the full PhotoGuard paper before writing code to understand the attack and assess my implementation. More than half of the semester's marks are individual, so I took ownership of the overall plan and became the team's research lead.

Week 3 (10–16 Aug) · Choosing the baseline by running it. We considered IMPRESS and the Query-Free (QF) attack suggested by my teammate. I ran both on a Colab T4 and asked the lecturer to confirm in writing that a baseline needed runnable code and replicable metrics. QF perturbs the text prompt, not the protected image, and does not attack PhotoGuard. I kept IMPRESS as the baseline, moved QF to Related Work, and explained the decision to my teammate so her suggestion still contributed. IMPRESS judged editing success only by eye, so I added a quantitative score (a CLIP-based edit-success rate). From M2 onward, the headline measure became the SSIM-based recovery `R_pipe` (Section I). My first attack plan placed cheap purifiers before IMPRESS's harness (JPEG, blur, resizing, and DiffPure [4]). I also analysed the frequency band of PhotoGuard's perturbation as a possible filter target. The masked method replaced both ideas in Week 6; Section IV-A returns to DiffPure.

Week 4 (17–23 Aug) · Making the baseline run and preparing M1. The 2023 IMPRESS release did not run as published. I tested it on Colab and repaired each failure, including a deleted model repository, a missing dependency, and a multi-GPU assumption. I read `impress.py` line by line and recorded which parts belonged to the authors and which were ours. This clarified that our work extended the code instead of claiming its core algorithm. I then designed our M1 challenge, the Tyro Wash Test. A robustness-only score could reward a challenger for making the photo unusable, so I added perceptual cost as a second axis. I finished the week by drafting our M1 Ed post.

Week 5 (24–30 Aug) · Submitting M1 and finding the real problem. I ran the final M1 pipeline on a Kaggle T4 after finding that the older P100 GPU was incompatible with the current PyTorch, then submitted on 28 Aug. Recovery was +5.2% at one protection setting and −1.2% at a stronger one. I published both results, including the unfavourable number. After submission, I refined the M2 code and recorded each test and output. One observation redirected the project: purification cost an LPIPS of 0.15, about nine times the damage of the protection it removed and above my 0.10 challenger budget. Reducing that cost became the M2 goal.

Week 6 (31 Aug–6 Sep) · Building a method I could defend. I measured regional changes during an edit to locate wasted damage. The kept region moved 4–7 grey levels; the repainted region moved 86–96. This led to masked purification (Eq.~(1)), a four-line change. I added a no-purification floor and a 1000-iteration IMPRESS control. The first small test exposed reversed mask polarity. After the fix, the smoke test cut photo damage from 0.160 (IMPRESS) to 0.037 (masked). An accidental second run established the noise band: photo damage reproduced exactly, while recovery moved 2–4 pp. I adopted two rules: verify image conventions through measurement and run the cheapest test first.

Week 7 (7–13 Sep) · Testing the measurement before the method. Before tuning the attack, I tested whether our evaluation could detect the defence at all. I added three controls: a seed floor (editing the clean image twice with different seeds), a matched random-noise baseline, and a rerun at twice the nominal protection strength. The editor's own seed-to-seed variation changed the output more than the protection did, so the baseline's metric could barely see the defence it was meant to evaluate. This was a more valuable finding than a small gain in recovery. I also found that the strength setting `pg_eps` had no effect at step size 1, and that the SSIM gate of 0.85 I had published for challengers accepted every case. I replaced it with a measured per-image floor and announced the correction publicly.

Week 8 (14–20 Sep) · Submitting early and choosing a target. I submitted M2 three days early, which left the rest of the week to prepare M3. On two faces, masked purification matched the 1000-iteration control on recovery (+14.5 vs +13.5 pp) at a quarter of IMPRESS's photo damage (0.038 vs 0.150). I then read other teams' M2 posts to choose the defence I would attack individually. I chose Cyber Ninjas' ESD-x erasure over the alternative I had considered, whose win condition required generating harmful content; ESD-x offered a clear, safe and testable question instead. I posted a falsifiable prediction on their thread before running any test, so I could not reshape the hypothesis after seeing the result. Applying my new gate to my own M2 images showed that neither had been affected by the protection, so strengthening the protection became the first task for M3. Finally, because my teammate's contributions had stopped, I informed the tutor and replanned every remaining deliverable in case I might complete it independently.

Mid-semester break (21–27 Sep) · One notebook, one question. I moved the canonical experiments to Colab, the platform required by the brief, and split them into separate notebooks. Each session answered one question and could survive a disconnect. All four purification versions shared one protection stage, removing the last inconsistency. The protection sweep from Week 8 closed: stronger protection cost 3.5× the photo damage (LPIPS 0.027 → 0.096) and put 4 of 10 faces over the 0.10 budget. The editor noticed it within budget on at most 1 of 10 faces. I kept the original setting and stopped tuning the defence. A review of PhotoGuard's reference code showed that our protection used only one-twenty-fifth of the reference optimisation effort. I stated this limitation before presenting results and narrowed the claim: masked purification reduced photo damage; recovery remained inconclusive.

Week 9 (28 Sep–4 Oct) · Accepting a refuted prediction. I ran the cross-script test against ESD-x, and my prediction was refuted: the unmodified model itself did not draw Van Gogh from the non-Latin names, so they could not bypass the erasure. I acknowledged this on Cyber Ninjas' thread and credited the part of their argument that had anticipated it. On the IMPRESS side, I realised that weeks of protection sweeps had focused on the defence and neglected my attack. I refocused by logging IMPRESS's optimisation loss and saving checkpoints. This turned the 100- and 1000-iteration versions into one observed training run. Over 1000 iterations, the loss fell 39% and photo damage fell from 0.181 to 0.143. Both flattened after about 500, producing the trade-off in Table I. At n = 10, masking's M2 recovery gain at n = 2 disappeared (+0.9 pp), while its damage advantage held.

Week 10 (5–11 Oct) · Making a failure informative. The tutor confirmed that an explained failed attack could still earn marks. I designed a second ESD-x diagnostic with a fixed reference, fresh seeds, and a Gauguin control. A review of the first test showed that the apparent "99% of the style survives" result came from a moving reference. Against a fixed reference, only 14% remained. The second test found no bypass. It showed that the erasure was specific and that CLIP scoring could miss visible motifs. I then built the M3 slide deck and speaking script.

Week 11 (12–18 Oct) · Using feedback as a design input. M2 feedback showed that I had explained *what* happened more clearly than *why*. I returned to the source code and training log. They linked the decrease in photo damage during longer optimisation to IMPRESS's initial random noise: the loss term that removes it falls from 0.29 to 0.12 over 1000 iterations, approaching its 0.10 limit. They also explained why masking preserves recovery: the editor discards the region we stopped

purifying. I rebuilt the M3 slides around mechanisms. I also stated plainly that no other team had attacked our project, avoiding any claim of an interaction that had not occurred.

Week 12 (19–25 Oct) · Coordinating the presentation and building the M4 demos. My teammate and I rehearsed the M3 presentation on Zoom and exchanged written scripts. Mine included cues for her slide changes. The scripts kept our delivery aligned and made each role visible. After M3, I built the two M4 demonstrations. A preliminary run caught detector data leakage before the full experiment. Both full demonstrations missed their pre-set criteria. I stopped as planned and reported the failures without changing the tests after seeing the results.

IV. Adversarial Gameplay

A. Unconstrained System Enhancement: Masked Diffusion Purification

Proposed enhancement. IMPRESS removes the protection by *optimisation*: 100 gradient steps per photo, each passing through the VAE. A diffusion model, however, is already trained to remove Gaussian noise, so the protection can instead be removed by *denoising*:

1. encode the protected photo and add a small amount of noise (strength 0.1: the noise level 10% of the way along the forward schedule, so only the last 5 of 50 denoising steps run);
2. let Stable Diffusion v1-5 denoise it for those 5 steps with an empty prompt and no guidance;
3. keep the result only in the kept region, using the mask of Eq. (1).

The added noise drowns the protective perturbation, and the denoiser has no reason to reconstruct it. This is DiffPure [4], originally a defence for classifiers, turned against an editing defence; it was the last stage of my Week 3 purifier plan. It is also the same mechanism as AudioPure [5] discussed in the Week 8 workshop [14], and the denoised-smoothing defence [11] discussed in Week 3 [14] used in reverse: a denoiser placed in front of the editor, but by the attacker. It requires no gradients and no per-image optimisation.

This change would improve the attack's practical scale. Cutting purification from about 107 s to 1 s turns costly per-photo optimisation into a batch operation. The proposed unconstrained system would combine this fast front end with full-strength PhotoGuard evaluation, a narrow strength search, and a short consistency-restoration stage. The demonstration tests the front end before adding those larger components.

Demo. `M4/Dark_Tyro_M4_demos.ipynb`, Part A. The notebook reuses the M3 pipeline and settings unchanged (the same ten faces, protection settings and editor), regenerates the protected photos, and adds version D at strengths 0.1 and 0.2. **Criterion, fixed before the run:** D succeeds if its photo damage is no higher than C's, its recovery is within 3 pp of C's, and its purification time is lower than A's. The run was limited to one session and two strengths; if D failed, the table would be reported as the result.

Re-run check. Because the protected photos were regenerated, A and C double as a replication of Table I. Photo damage reproduced (A 0.185 $\rightarrow$ 0.184, C 0.060 $\rightarrow$ 0.059), but recovery moved by 4–5 pp (A +0.2 $\rightarrow$ -5.1, C +0.9 $\rightarrow$ -3.2), wider than the ±3 pp band measured on identical inputs. A tiny change in the protected photo is amplified by the editor, the same sensitivity PhotoGuard relies on. This is a further reason to judge D on damage and time.

Verdict: fails the damage criterion. D at strength 0.1 caused 19% more photo damage than C (0.070 vs 0.059). It passed the other two checks: its recovery was 1.2 pp above C's (within noise), and it was 93$\times$ faster than A. Doubling the strength worsened both metrics (damage +44% relative to C). Against the **IMPRESS baseline**, D at 0.1 caused **62% less photo damage** in about 1% of the purification time, with recovery within noise.

Analysis. - *D does not beat C.* Denoising regenerates the kept region from the model's prior, so it alters genuine detail as well as the perturbation. C changes the image only as far as IMPRESS's loop requires. Higher strength regenerates more of the photo, which is why 0.2 performs worse than 0.1. - *Recovery is uninformative here.* Every recovery value is negative or within noise, and one face (221629697) scores -37 pp under A. The protection is as weak as in M3, so recovery has almost nothing to measure; photo damage and purification time are the reliable metrics. - *Extension without resource constraints.* The proof of concept establishes speed, not superiority over C. With unrestricted time and compute, I would run PhotoGuard at its full 200 $\times$ 10 reference effort to make recovery discriminative; sweep diffusion strength between 0.05 and 0.1; evaluate more faces, prompts, masks, and editors; and add a short IMPRESS-style consistency stage after denoising. This hybrid would use D to remove most of the perturbation quickly and reserve expensive optimisation for the residual error.

B. Role Reversal: Defending PhotoGuard

Had I been on the opposite side, I would have defended PhotoGuard, the defence our project attacked; ESD-x belonged to a different team. Following the layered anti-deepfake design discussed in the Week 5 workshop [14], I would use the following complementary strategies, because no single defence remains reliable after the attacker adapts:

1. **Purification-aware protection.** Optimise the perturbation *through* the purifier, averaging over random purifier settings (expectation over transformation [13]), so that the protection is built to survive the attacker's next move. The cost is that the protection must differentiate through a purifier.
2. **Concentrate the budget where the editor reads.** Our M2 finding applies to the defender as well: the editor blanks the region it repaints, so perturbation placed there is wasted. A defender who can anticipate common edit regions (background, clothing) should concentrate the perturbation in the region the editor keeps.
3. **Make the protection strong enough to take effect.** Our Week 7 finding and the Week 2 material on gradient-sign attacks [10] point the same way: at step size 1 the optimiser never reaches its eps budget, so raising

TABLE II
HOW THE RESULTS DEVELOPED ACROSS THE SEMESTER

Week	Experiment (n)	Key result	What it changed
5	M1 pipeline (2 faces, 2 protection settings)	Recovery $+5.2\%$ / -1.2%; purification damage 0.15 vs protection ≈ 0.02	Reduce purification damage $\rightarrow$ M2
6	M2 smoke test, run twice (2 faces)	Damage $0.160 \rightarrow 0.037$ with mask; recovery moved 2–4 pp between identical runs	Report damage as a point, recovery as a range
8	M2 four versions (2 faces)	Recovery N 0 / A $+9.5$ / B $+13.5$ / C $+14.5$ pp; damage 0.017 / 0.150 / 0.108 / 0.038	Gate: neither face was affected by the protection $\rightarrow$ strengthen it, raise n
8–break	Protection sweep (10 faces)	Noticed within budget on $\leq$ 1 of 10 at $3.5\times$ protection damage	Keep setting; stop tuning the defence
9	M3 training run (10 faces)	C -68% damage vs A; B $+4.9$ pp recovery, carried by 2 faces	Claim a trade-off, not a win
9–10	ESD-x tests (120 + 84 images)	No bypass; description route $14\% \rightarrow -4\%$; Gauguin 105%	Explain the failure instead of hunting further
12	M4 demos (10 faces)	Tables III–IV: both criteria missed	Report as measured; stop

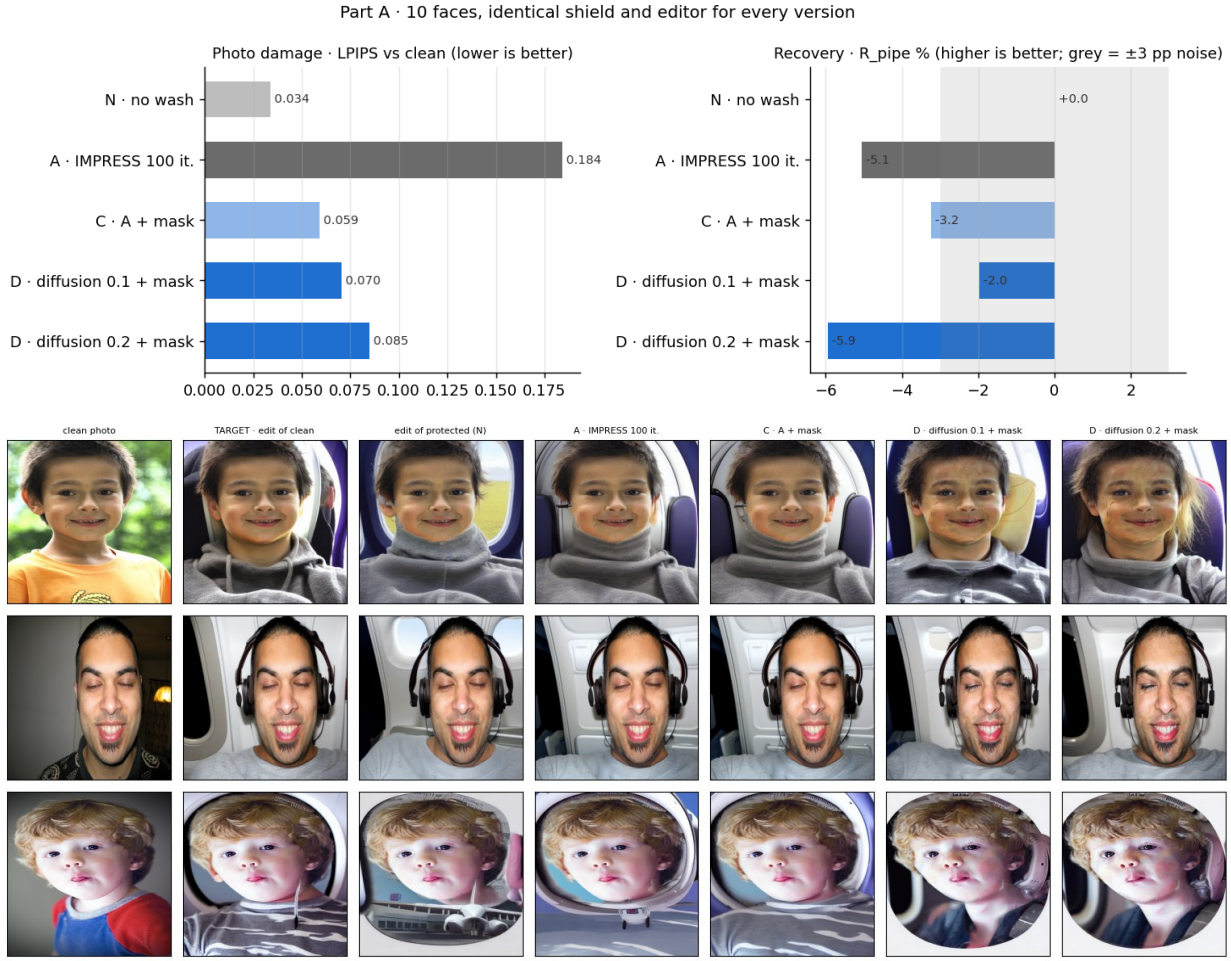


Fig. 2. Part A. Top: photo damage and recovery for each version (grey band: ± 3 pp noise). Bottom: edits of three faces under every version.

`pg_eps` changes nothing. A defender should check the step size before raising eps.

4. **Detect purified photos** (demonstrated below). If removing the protection leaves a fingerprint, a platform can refuse to edit, or flag, a photo that appears purified.
5. **Provenance.** A signed or watermarked original (the preventive layer from Week 5) cannot stop the attack. It can prove that the purified copy differs from the original.

Proof of concept: detecting purified photos. Part B of the same notebook demonstrates the fourth layer.

Feature, derived from how the attack works. IMPRESS minimises one quantity: how much a photo changes on a round trip through the VAE. I expected a purified photo to be *more* VAE-consistent than any natural photo, leaving a fingerprint of the attacker's objective. The feature is the VAE round-trip error over 32×32 patches. It uses the **10th-percentile patch**, so that purifying only part of the photo would still register.

Avoiding leakage. The threshold was set on 40 held-out

TABLE III
PART A RESULT (RUN 23 OCT, $n = 10$)

Version	Damage ↓	Rec. (pp) ↑	$> +3$ / < -3	s/face ↓
N · no purification	0.034	0.0	–	0
A · IMPRESS 100	0.184	−5.1	2 / 5	107.1
C · A + mask	**0.059**	−3.2	3 / 4	107.1
D · diff. 0.1 + mask	0.070	−2.0	3 / 5	**1.15**
D · diff. 0.2 + mask	0.085	−5.9	3 / 6	1.60

TABLE IV
PART B RESULT (RUN 23 OCT)

Group	n	Median	Flagged	AUC*
clean, held-out (calibration)	40	0.39	2	–
clean, test	10	0.40	0	–
protected (N)	10	0.39	0	0.48
purified · A (IMPRESS)	10	**3.57**	0	0.00
purified · C (A + mask)	10	0.40	0	0.47
purified · D 0.1	10	0.40	0	0.51
purified · D 0.2	10	0.40	0	0.51

Score: VAE round-trip error of the 10th-percentile 32×32 patch ($\times 1000$). Flagged: pre-set rule. *AUC in the direction fixed in advance (lower score = purified); 0.5 = chance; 0.00 = perfect separation in the *opposite* direction.

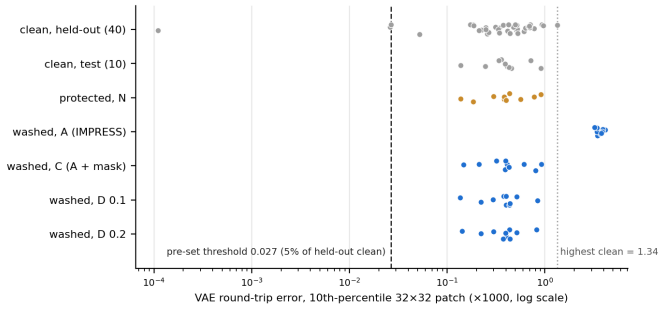


Fig. 3. Part B. Detector score of every photo by group, with the pre-set threshold (log scale).

clean faces that never appear in the test set: a photo is flagged if its score falls below the 5th percentile (0.027), giving a 5% false-alarm rate. **Criterion, fixed before the run:** at least 8 of 10 photos flagged for at least one purification method.

Verdict: fails. The detector, as specified in advance, flagged none of the purified photos.

Observation (found after the run). IMPRESS-purified photos deviate in the opposite direction. Their scores (3.23–4.16) are about nine times the clean median, and every one exceeds the highest score among all 50 clean photos (1.34). Any threshold between 1.34 and 3.23 would separate all 10 purified photos from all 50 clean ones. Because this direction was chosen after seeing the data, I report it as an observation that requires confirmation on new faces with a threshold fixed in advance.

Analysis. The most likely explanation for A's unexpectedly high error is IMPRESS's initialisation, although this mechanism remains to be tested. IMPRESS adds random noise to the photo (`pur_noise`) and then optimises it away; our M3 training log shows that the loss term which removes this noise is still 0.17 after 100 iterations, well above its 0.10 limit, and approaches it (0.12) only after about 500. Because the VAE cannot reproduce random noise, the round-trip error rises across the photograph, including in its most consistent patches. A cheap test follows: the 1000-iteration photos saved in M3 (version B) should score much closer to clean.

C and D do not show this fingerprint because the mask restores protected pixels over most of the photograph. The 10th-percentile patch falls in an unpurified region. Our M2 method already evades this detector, revealing the attacker's cheapest counter-move: purify less of the photograph. A defender should score only the region an editor is likely to keep and combine detection with purification-aware protection. If the protection survives local purification, the attacker must use whole-image purification—the operation this feature detects most clearly.

The detector also assumes access to the attacker's VAE. This is plausible for Stable Diffusion 1.x, whose VAE is public, but a different editor would require a matched detector. More importantly, the reversed signal and its threshold must be tested on new data before either can support a detection claim.

C. Personal Reflection and Background

I entered this unit with a data science background and little experience in machine-learning security, PyTorch, or generative models. My experience in exploratory analysis shaped the strategy. I trusted measurements over explanations, added a no-purification arm, a seed floor, and a held-out calibration set, and fixed each pass/fail criterion before running. I also recomputed results from saved outputs instead of relying on notes. Three useful moments corrected my own published claims: the reversed mask, the incorrect SSIM gate, and the "99% survives" artefact. Each correction came from a deliberate control.

This knowledge gap led me to rely heavily on AI assistants. In the first weeks, their suggestions shaped much of the project because I could not yet assess each method. Weekly workshops and labs changed this. After learning Stable Diffusion, its VAE, cross-attention, and the noise schedule, I could test suggestions against the code and data. I also began rejecting weak suggestions. AI use was permitted for this task. Its value depended on my ability to verify its output.

Looking back over M1–M3, working with IMPRESS taught me that both sides of this theme meet in one component: Stable Diffusion's VAE. PhotoGuard pushes the photo's VAE encoding off course; IMPRESS undoes this by asking the same VAE to agree with the photo again, minimising the round-trip error $\|x - D(E(x))\|$ while an LPIPS bound keeps the result close to its input. Reading its 25 lines also showed me what this design costs. The loop starts by adding random noise and needs hundreds of iterations to remove it, which is why photo damage fell with longer training. It purifies the whole photo, although the inpainting editor discards the region it repaints. And, as Part B suggests, it can leave its own trace in the VAE's round-trip error. Each property became one of my strategies: masked purification answers the second, diffusion purification replaces the slow loop, and the detector targets the third.

Working largely alone after M1 also taught me to plan every deliverable so that one person could complete it, and

to set stopping rules before open-ended experiments. In the final week, written scripts proved to be an effective way to coordinate with my teammate.

V. Limitations and Conclusion

Limitations.

1. Throughout the semester, protection ran at one-twenty-fifth of PhotoGuard's reference effort. Recovery had little to distinguish and shifted by 4–5 pp when the protection stage was re-run (Section IV-A). Photo damage was the reliable metric.
2. The evaluation used 10 faces, one prompt, one editor and one inpainting mask. The 5% false-alarm rate was estimated from 40 faces, so it carries wide error bars.
3. D's purification time excludes the one-off model loading time, whereas IMPRESS's time includes its own.
4. The direction of the Part B observation was chosen after seeing the data, and the survival of Van Gogh motifs under ESD-x was checked by eye on a single seed.

Conclusion. My most productive attacks targeted measurements: an evaluation that barely detected its own defence, an erasure judged against a moving reference, and a purification step that caused more damage than the protection. Both M4 demonstrations missed their pre-set criteria, but each produced a clear finding. Compared with IMPRESS, diffusion purification reduced damage by 62% and ran 93$\times$ faster. It did not beat the masked variant. Whole-photo IMPRESS purification left a strong VAE fingerprint; masked purification did not. Each move allows a relatively cheap counter-move. The strongest defensive investment is protection trained to survive purification, the layer that costs an attacker most to overcome.

References

[1] H. Salman, A. Khaddaj, G. Leclerc, A. Ilyas, and A. Madry, "Raising the cost of malicious AI-powered image editing," in *Proc. ICML*, 2023.

[2] B. Cao, C. Li, T. Wang, J. Jia, B. Li, and J. Chen, "IMPRESS: Evaluating the resilience of imperceptible perturbations against unauthorized data usage in diffusion-based generative AI," in *Proc. NeurIPS*, 2023.

[3] R. Gandikota, J. Materzyńska, J. Fiotto-Kaufman, and D. Bau, "Erasing concepts from diffusion models," in *Proc. ICCV*, 2023.

[4] W. Nie, B. Guo, Y. Huang, C. Xiao, A. Vahdat, and A. Anandkumar, "Diffusion models for adversarial purification," in *Proc. ICML*, 2022.

[5] S. Wu, J. Wang, W. Ping, W. Nie, and C. Xiao, "Defending against adversarial audio via diffusion model," in *Proc. ICLR*, 2023.

[6] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proc. CVPR*, 2022.

[7] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in *Proc. CVPR*, 2018.

[8] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Trans. Image Process.*, vol. 13, no. 4, pp. 600–612, 2004.

[9] A. Radford *et al.*, "Learning transferable visual models from natural language supervision," in *Proc. ICML*, 2021.

[10] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," in *Proc. ICLR*, 2015.

[11] H. Salman, M. Sun, G. Yang, A. Kapoor, and J. Z. Kolter, "Denoised smoothing: A provable defense for pretrained classifiers," in *Proc. NeurIPS*, 2020.

[12] V. Le, J. Brandt, Z. Lin, L. Bourdev, and T. S. Huang, "Interactive facial feature localization," in *Proc. ECCV*, 2012.

[13] A. Athalye, L. Engstrom, A. Ilyas, and K. Kwok, "Synthesizing robust adversarial examples," in *Proc. ICML*, 2018.

[14] FIT5230 Malicious AI workshop slides, Weeks 2, 3, 5, 8 and 10, Monash University, 2026.